

# INTRODUCTION TO EMBEDDED SYSTEMS & IoT

## What is an Embedded systems?

An Embedded System is a microcontroller or microprocessor-based system designed to control, monitor, or assist the operation of electronics devices or machines.

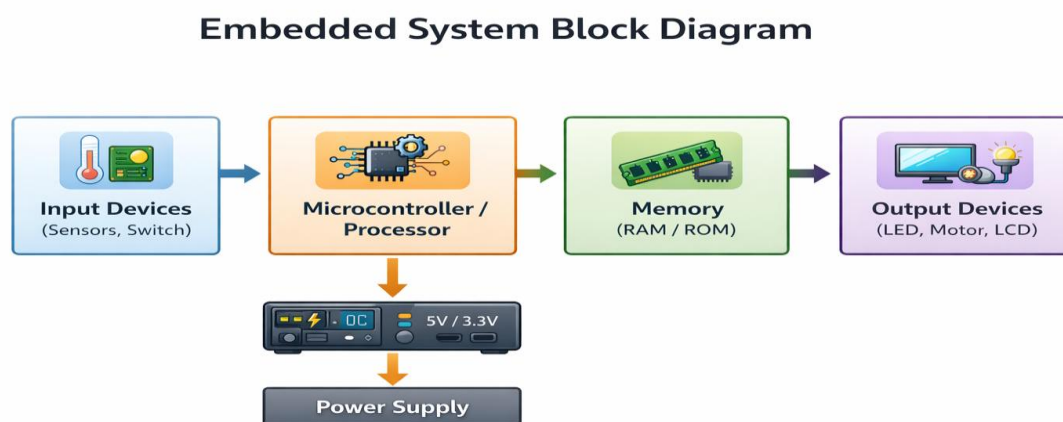
### Key Characteristics:

- Dedicated function (designed for a specific function).
- Real-Time operation (responds within a fixed time).
- Low power consumption.
- Compact Size.
- Reliable and efficient.

### Components of an Embedded System:

1. Microcontroller / Microprocessor – main processing unit
2. Memory (RAM, ROM, Flash) – stores data and program
3. Input Devices – sensors, switches, keyboards
4. Output Devices – displays, LEDs, motors, actuators
5. Software / Firmware – program that controls the system

### Block diagram of Embedded Systems:



- 1.**Input Devices:** Sensors and switches collect real-world data such as temperature, motion, or button presses.
- 2.**Microcontroller/Processor:** Acts as the brain of the embedded system. It processes input data and executes the embedded software.
- 3.**Memory:** Stores the firmware and runtime data.
  - **ROM/Flash:** stores program code.
  - **RAM:** stores temporary data during execution.
- 4.**Output Devices:** Displays the result or performs actions using LEDs, displays, motors, buzzers, etc.
- 5.**Power Supply:** Provides regulated power (usually 3.3V or 5V ) to all system components.

#### **Advantages of Embedded Systems:**

1. **Small Size** – Embedded systems are compact and require less space.
2. **Low Power Consumption** – They consume very little electrical power.
3. **High Efficiency** – Designed for specific tasks, so they work faster and efficiently.
4. **High Reliability** – Less hardware complexity increases reliability.
5. **Cost Effective** – Mass production reduces overall cost.
6. **Fast Operation** – Performs tasks quickly because it is dedicated to one function.

### **Disadvantages of Embedded Systems:**

1. **Limited Functionality** – Designed only for a specific task.
2. **Difficult to Upgrade** – Hardware and software upgrades are not easy.
3. **Limited Memory** – Storage capacity is usually small.
4. **Complex Design Process** – Designing embedded systems requires specialized skills.
5. **Debugging is Difficult** – Troubleshooting hardware and software together can be challenging.

### **Real-World Application of Embedded Systems:**

- Smart home automation systems
- Smart lighting systems
- Washing machines
- Microwave ovens
- Smart TVs
- Engine Control Unit (ECU) in cars
- Anti-lock Braking System (ABS)
- Airbag control systems
- Heart rate monitors

## Difference between Microcontroller and Microprocessor:

| Feature           | Microprocessor                                                                   | Microcontroller                                                                                    |
|-------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Definition        | A microprocessor is a CPU used for general-purpose computing.                    | A microcontroller is a compact system that includes CPU, memory, and peripherals on a single chip. |
| Components        | Contains only the processor (CPU). External memory and I/O devices are required. | Contains CPU, RAM, ROM, I/O ports, timers, and peripherals in one chip.                            |
| Power Consumption | Consumes more power.                                                             | Consumes less power.                                                                               |
| Cost              | Higher cost due to external components.                                          | Lower cost because everything is integrated in one chip.                                           |
| Application       | Used in computers and laptops for complex tasks.                                 | Used in embedded systems for specific control tasks.                                               |
| Examples          | Intel Pentium, Intel Core processors                                             | Arduino (ATmega328), PIC, 8051                                                                     |

## Role of Microcontrollers (Arduino, ESP32, Raspberry Pi Pico)

Microcontrollers are essential components in embedded systems used to control and manage electronic devices. They integrate a CPU, memory, and input/output interfaces on a single chip to perform dedicated tasks.

### 1. Arduino

Arduino is an open-source microcontroller board widely used for embedded system development and prototyping.

**Key Features:**

- Based on microcontrollers like ATmega328
- Digital and Analog input/output pins
- Easy programming using Arduino IDE
- Supports sensors, motors, and displays

**Role in Embedded Systems:**

- Reads sensor data
- Processes data using programmed instructions
- Controls actuators like LEDs, motors, and relays

**2. ESP32**

ESP32 is a powerful microcontroller developed for IoT and wireless communication applications.

**Key Features:**

- Built-in **Wi-Fi and Bluetooth**
- Dual-core processor
- Low power consumption
- Multiple GPIO pins and communication interfaces

**Role in Embedded Systems:**

- Connects devices to the internet
- Enables wireless communication between devices
- Controls sensors and actuators in smart systems

### 3. Raspberry Pi Pico

**Raspberry Pi Pico** is a microcontroller board based on the **RP2040 chip** designed for high performance and low cost.

#### **Key Features:**

- Dual-core ARM Cortex-M0+ processor
- 26 GPIO pins
- Supports **MicroPython and C/C++ programming**
- Low power consumption

#### **Role in Embedded Systems:**

- Controls hardware components
  - Processes sensor data
  - Used in automation, robotics, and industrial control systems
- Microcontrollers such as Arduino, ESP32, and Raspberry Pi Pico act as the core components of embedded systems, enabling device control, data processing, and communication in various applications like IoT, robotics, and automation.

## **Basic Sensors in Embedded Systems:**

Sensors are devices used in embedded systems to detect physical changes in the environment and convert them into electrical signals that a microcontroller can process.

### **1. Temperature Sensor**

A Temperature sensor measures the temperature of the surrounding environment and converts it into an electrical signal.

**Examples:** LM35, DHT11, DHT22

#### **Applications:**

- Weather monitoring systems
- Smart home temperature control
- Industrial temperature monitoring
- Medical devices

### **2. Motion Sensor**

A motion sensor detects movement of objects or people within a specific area.

**Example:** PIR (Passive Infrared) Sensor

#### **Applications:**

- Security alarm systems
- Automatic lighting systems
- Smart home automation
- Surveillance systems

### **3. Light Sensor**

A light sensor detects the intensity of light in the environment.

**Examples:** LDR (Light Dependent Resistor), Photodiode

#### **Applications:**

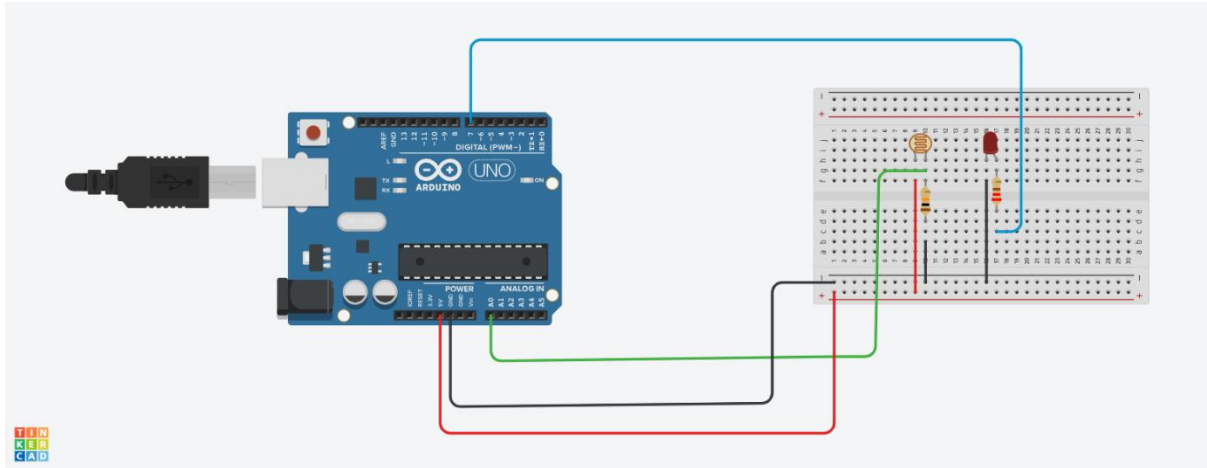
- Automatic street lighting
  - Smartphone brightness control
  - Solar tracking systems
  - Smart lighting systems
- 
- Temperature, motion, and light sensors are basic components in embedded systems that collect environmental data and help microcontrollers control devices automatically.



### **Working Principle of a Smart Lighting System:**

- The smart lighting system uses a Light Dependent Resistor (LDR) to detect the intensity of light in the surrounding environment.
- The resistance of the LDR changes depending on the light intensity.
  - In bright light, the resistance of the LDR decreases.
  - In low light or darkness, the resistance of the LDR increases.
- The change in resistance of the LDR produces a change in voltage in the circuit through a voltage divider arrangement.
- This varying voltage signal is given to the Analog input pin of the microcontroller.
- The microcontroller reads the sensor signal and converts the Analog voltage into a digital value using its internal Analog-to-Digital Converter (ADC).
- The microcontroller continuously monitors the sensor value and compares it with a predefined threshold value stored in the program.
- If the sensor value indicates low light intensity (dark condition), the microcontroller sends a HIGH signal to the output pin connected to the LED.
- When the output pin becomes HIGH, current flows through the LED, causing the LED to turn ON.
- If the sensor value indicates high light intensity (bright condition), the microcontroller sends a LOW signal to the output pin.
- When the output pin becomes LOW, current flow to the LED stops, causing the LED to turn OFF.
- This process runs continuously, allowing the system to automatically control the lighting based on the surrounding light conditions.

## Circuit Connection of Smart Lighting System Using LDR Sensor and Arduino for Automatic LED Control:



### Arduino Code for Smart Lighting System (LDR + LED):

#### Program (C++):

```
const int ledPin = 7;
const int ldrPin = A0;

void setup(){

  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  pinMode(ldrPin, INPUT);
}

void loop(){
  int ldrStatus = analogRead(ldrPin);

  Serial.println(ldrStatus);

  if (ldrStatus <=80){
    digitalWrite(ledPin, HIGH);
  }
  else {
    digitalWrite(ledPin, LOW);
  }
}
```

## Code Explanation:

### 1. Declaration:

```
const int ledPin = 7;  
const int ldrPin = A0;
```

- ledPin represents the digital pin connected to the LED.
- ldrPin represents the analog pin connected to the LDR sensor.
- const int means the pin numbers remain constant throughout the program.

### 2. Setup Function:

```
void setup()  
{  
  Serial.begin(9600);  
  pinMode(ledPin, OUTPUT);  
  pinMode(ldrPin, INPUT);  
}
```

The setup() function runs only once when the Arduino starts.

Functions inside setup

Serial.begin(9600);

- Starts serial communication between Arduino and the computer.
- Used to display sensor values in the Serial Monitor.
- 9600 represents the baud rate (data transmission speed).

```
pinMode(ledPin, OUTPUT);
```

- Sets pin 7 as an output pin because the LED is controlled by the Arduino.

```
pinMode(ldrPin, INPUT);
```

- Sets analog pin A0 as input because it receives the signal from the LDR sensor.

### 3. Loop Function:

```
void loop()
```

The loop() function runs continuously, allowing the system to monitor the sensor and control the LED automatically.

### 4. Reading the Sensor Value:

```
int ldrStatus =analogRead(ldrPin);
```

- analogRead() reads the voltage from the LDR sensor.
- Arduino converts the analog signal into a digital value between 0 and 1023 using the ADC (Analog to Digital Converter).
- The value is stored in the variable ldrStatus.

Example:

- Dark environment → smaller value
- Bright environment → larger value

### **5.Displaying Sensor Value:**

```
Serial.println(ldrStatus);
```

- Prints the LDR value in the Serial Monitor.
- Helps in checking and adjusting the light threshold value.

### **6.Condition Check (Decision Making):**

```
if (ldrStatus <= 80)
```

- The program checks whether the sensor value is less than or equal to 80.
- This value represents the darkness threshold.

### **7. LED Turns ON:**

```
digitalWrite(ledPin, HIGH);
```

- Sends HIGH signal (5V) to the LED pin.
- Current flows through the LED.
- LED turns ON when the environment is dark.

## **8. LED Turns OFF:**

```
digitalWrite(ledPin, LOW);
```

- Sends LOW signal (0V) to the LED pin.
- Current flow stops.
- LED turns OFF when the environment is bright.

## **Overall Working of the Code:**

1. The LDR sensor detects the surrounding light intensity.
2. Arduino reads the sensor value using `analogRead()`.
3. The value is compared with a predefined threshold.
4. If the environment is dark, the LED turns ON.
5. If the environment is bright, the LED turns OFF.
6. The process repeats continuously inside the `loop()` function.

**This program automatically controls the LED based on surrounding light conditions using an LDR sensor and Arduino.**

## **Real-World Applications of Smart Lighting System :**

- **Street Lights:** Smart lighting is used in automatic street lights. The lights turn ON at night and OFF during the day using a light sensor.
- **Home Lighting:** In houses, smart lighting systems automatically turn lights ON when it becomes dark and OFF when there is enough daylight.
- **Garden Lighting:** Gardens and parks use smart lights that automatically glow in the evening to provide light for people walking.
- **Parking Areas:** Parking lots in malls and offices use automatic lighting so that lights turn ON at night for better visibility and safety.
- **Security Lighting:** Smart lighting is used near gates, buildings, and warehouses to improve security by providing light during dark conditions.
- **Public Places:** Places like bus stops, campuses, and pathways use smart lighting systems to save electricity by turning lights ON only when needed.